

A Survey and Controlled Synthetic Retrieval Study of Efficient Language Models

Jeffrey Huang

University of California, Los Angeles

May 2026

1 Introduction

Long-context expectations for large language models have grown significantly: source files, research papers, transcripts, retrieved documents, and extended conversations are now commonly presented to a model as part of a single prompt. The dominant decoder-only Transformer architecture is well suited to such settings because self-attention gives each token a direct mechanism for accessing earlier tokens in the sequence [1]. This explicit access is one reason Transformers perform well on retrieval-like behavior: when a later token must recover a fact stated earlier in the prompt, attention can, in principle, route information directly from the relevant earlier position.

The same mechanism is also the source of the long-context bottleneck. In standard self-attention, pairwise interactions are computed between positions in the sequence. During training, this creates a quadratic dependence on context length, while autoregressive inference requires storing and reading a growing key-value cache. This cost is manageable for short contexts, but as prompt length grows, the cache becomes increasingly memory- and bandwidth-intensive. As a result, a sizable body of recent work explores whether the useful behavior of attention can be approximated, compressed, or partially replaced by more efficient sequence mixing mechanisms.

This paper surveys several representative approaches to that problem and then tests selected public checkpoints on controlled synthetic key-value retrieval. The survey is organized around a central trade-off. Although full attention preserves explicit token-level access to the prompt, it is expensive. Linear, recurrent, state-space, and convolutional alternatives reduce this cost by representing the past through a structured or fixed-size state. That compression improves efficiency, especially during inference, but it may weaken precise retrieval when many similar facts compete within a finite memory space. Needle-in-a-haystack tasks provide a useful probe of this tradeoff because they isolate a simple question: can the model recover a specific key-value association from a long, mostly irrelevant context? The empirical study uses this controlled slice of long-context retrieval to study two public checkpoint families: a HazyResearch/BASED family comparison involving attention, Mamba, and BASED models, and a larger State Spaces comparison of Mamba, Mamba-2, and Mamba-2-Attn.

2 The Long-Context Bottleneck

In a Transformer layer, an input sequence representation

$$X \in \mathbb{R}^{n \times d}$$

is projected into query, key, and value matrices,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Here, n is the number of tokens in the sequence and d is the hidden dimension of each token representation. The matrices Q , K , and V contain, respectively, the query, key, and value vectors for each token position: *queries* determine what each position is looking for, *keys* determine what each position offers as an address, and *values* contain the information that will be mixed into the position's output.

Causal self-attention computes

$$\text{Attn}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V,$$

where d_k is the key dimension and M is a causal mask that prevents a position from attending to future tokens. The matrix $QK^\top$ contains all pairwise query-key scores: entry (t, i) measures how strongly token t should attend to token i . Because this score matrix contains one entry for each ordered pair of positions, its size grows as n^2 with the sequence length. This quadratic structure is computationally expensive, but it also gives attention its distinctive modeling strength: each token can form a content-dependent connection to any earlier token.

Autoregressive inference changes the computational form but not the basic length dependence. During generation, the *prefix* is the sequence already available to the model: the original prompt plus any tokens generated so far. A decoder does not usually recompute keys and values for this entire prefix at each generation step. Instead, it stores them in a key-value cache. When the next token is generated, the model forms a new *query* for the current position and compares it against the cached *keys* for all earlier positions, then mixes the corresponding cached *values*. This avoids redundant recomputation over old tokens, but the cache still grows with context length and each new token must read from a longer memory. Long contexts therefore pressure both computation and memory movement.

Efficient long-context architectures can be interpreted as different responses to this bottleneck. Some retain attention but restrict which positions can interact. Others replace attention with recurrent state, state-space dynamics, linearized attention, long convolutions, or hybrid combinations of these mechanisms. The central question for retrieval is not only whether these models are asymptotically cheaper, but also how much information is retained when the full prompt is compressed into a smaller or more structured memory.

The following schematics describe how one sequence-mixing layer carries context through a single input sequence. Let x_t denote the hidden representation entering the layer at position t , and let y_t denote the representation produced after context has been mixed into that position. In a decoder language model, these representations are passed through later layers and projected to vocabulary logits for next-token prediction.

Full attention preserves the past as a token-indexed memory:

$$y_t = \sum_{i \leq t} \alpha_{t,i} v_i, \quad \alpha_{t,i} \propto \exp(q_t^\top k_i),$$

where q_t is the query at the current position, k_i and v_i are the key and value vectors stored for an earlier position, $\alpha_{t,i}$ is the attention weight assigned to that position, and y_t is the resulting output representation. As shown, for a single current position t , attention computes a weighted sum over previous value vectors. To achieve this, each earlier token remains separately addressable through its own key and value, meaning the model must store and read memory whose size grows with the prefix.

Compressed-memory models replace this growing token-indexed memory with a fixed-size state that is updated as the sequence is read,

$$s_t = F_\theta(s_{t-1}, x_t), \quad y_t = G_\theta(s_t, x_t).$$

In this schematic form, s_t is the memory state after reading token x_t , and F_θ and G_θ are learned update and readout functions. The sequence is processed from left to right: x_t updates the running state, and y_t is read from that updated state together with the current input. Information from the prefix is folded into s_t , creating efficient decoding while making interference between stored facts possible.

Linear-attention and fast-weight views use a related form of compression: they store associations between keys and values in an accumulated key-value statistic, such as

$$S_t = S_{t-1} + \phi(k_t) v_t^\top, \quad y_t \propto \phi(q_t)^\top S_t,$$

where $\phi(\cdot)$ is a feature map applied to keys and queries. The feature map makes key-query comparisons compatible with an accumulated summary: S_t stores many key-value associations, and the transformed query $\phi(q_t)$ reads from that shared statistic. This keeps the memory compact, but many associations must share the same finite representation.

Selective or gated recurrent models make the update itself depend on the current token, as in

$$s_t = (1 - z_t) \odot s_{t-1} + z_t \odot \tilde{s}_t,$$

where z_t is an input-dependent gate, $\tilde{s}_t$ is a candidate state update, and $\odot$ denotes element-wise multiplication. Gate values near zero preserve the previous state, while values near one write the candidate update; in typical neural implementations, this decision can vary across dimensions of the state. These forms provide the vocabulary used below: indexed lookup, compressed state, accumulated associations, and gated writing. Real architectures often combine these patterns, but the distinctions help organize how different models carry prior context forward.

3 Taxonomy of Efficient Long-Context Models

Efficient long-context models can be grouped by how they represent prior context and by how they reduce the cost of full attention. Table 1 organizes representative model families along those two axes. The term “efficient” is used broadly here: some models are strictly recurrent or linear-time in their sequence-mixing layer, while others reduce the cost of full attention through sparsity, locality, convolution, or hybridization. Practical efficiency also depends on constants, kernels, model width, batching, and hardware, so the table emphasizes the strategy each model uses to reduce the attention bottleneck.

Category	Representative models	Context representation	Efficiency strategy	Retrieval tradeoff
Full explicit access	Transformer	Stores token-indexed keys and values for the full prefix.	Pays full attention cost: $O(n^2)$ prefill/training and an $O(n)$ KV-cache read per generated token.	Strongest direct retrieval mechanism, but memory and bandwidth costs grow with context length.
Restricted explicit access	BigBird; local or sliding attention	Keeps attention, but limits which token pairs can interact.	Reduces the attention graph through sparse, local, global, or random patterns [2].	Retains some direct access, but retrieval depends on whether the pattern gives a usable path to the relevant token.
Compressed recurrent / state-space memory	RetNet, RWKV, Mamba, Mamba-2	Summarizes the prefix into recurrent, retention, or state-space state [3, 4, 6, 7].	Uses linear scans, parallel recurrent forms, or constant-state decoding instead of a growing KV cache.	Efficient long-context processing, but finite state can forget, overwrite, or merge similar facts.
Associative / fast-weight memory	Linear Transformers, Gated DeltaNet	Shared matrix-like key-value statistics [8, 9].	Linearizes attention-like computation or updates fast weights recurrently.	Stores key-value associations efficiently, but finite shared memory can suffer collisions or overwrite when many similar bindings are inserted.
Structured convolutional mixing	Hyena	Mixes long-range information through implicit filters and gating [5].	Uses long convolutional structure, often near-linear or $O(n \log n)$, rather than all-pairs attention.	Can propagate long-range signals through learned filters, but retrieval is mediated by structured mixing rather than direct token-indexed lookup.
Hybrid efficient architectures	BASED, Mamba-2-Attn, Griffin, Jamba, Nemotron-H	Combines partial attention with linear, recurrent, state-space, convolutional, or local sequence mixing [10, 7, 11, 12, 13].	Pays for explicit access only in selected layers, windows, or components while using cheaper mixers elsewhere.	May protect key-value binding where attention is retained, but behavior depends on the placement, frequency, and scope of those attention components.

Table 1: Representative strategies for efficient long-context language modeling. The categories are organized by how prior context is represented, how the architecture tries to reduce the cost of full attention, and what retrieval weakness that strategy can introduce.

These categories form a continuum of memory behavior. Several models combine mechanisms or admit multiple equivalent interpretations, but the grouping highlights the dominant way each family represents prior context. At one end, full attention preserves explicit token-level access at high cost.

Restricted-attention models such as BigBird keep the attention operation but reduce the set of token pairs that interact, often using local windows, global tokens, or sparse random connections to lower the cost of the attention graph [2].

Recurrent and state-space models move further toward compression. RetNet uses a retention mechanism with both parallel and recurrent forms, allowing the model to resemble attention for parallelizable training while supporting recurrent-style inference [3]. RWKV also pursues Transformer-like language modeling through recurrent updates: instead of storing a key-value cache for all prior tokens, it carries a compact state forward through the sequence [4]. Mamba introduces *input-dependent* selective state updates, so the model can vary what it writes into memory based on the current token [6]. Mamba-2 keeps the state-space perspective but reorganizes the computation through *state-space duality*, improving the architecture’s parallel and hardware-friendly forms [7].

Other categories compress or structure context in different ways. Hyena uses implicit long convolutions and gating, learning long filters that mix information across distant positions without full pairwise attention [5]. Linear attention methods rewrite attention-like computation so that key-value information can be accumulated into compressed statistics, a view closely related to fast-weight programming [9]. Gated DeltaNet extends this associative-memory direction with a gated delta-rule update intended to reduce overwrite and interference among similar associations [8]. Hybrid systems such as BASED, Mamba-2-Attn, Griffin, Jamba, and Nemotron-H occupy the middle of the design space: they retain some attention or local explicit access while relying on cheaper linear, recurrent, state-space, or convolutional components elsewhere [10, 7, 11, 12, 13].

This taxonomy sets the context for the empirical study, which samples two public checkpoint families from the broader design space. Experiment 1 compares full attention, compressed state-space memory, and hybrid linear attention within the HazyResearch/BASED family. Experiment 2 then zooms in on the State Spaces family, comparing original Mamba, Mamba-2, and a Mamba-2 hybrid with attention blocks.

4 Mechanistic Themes

The taxonomy describes how efficient models represent context. The same distinctions also suggest retrieval failure modes. Several recurring themes connect the model families above to synthetic key-value tasks. The first is the distinction between explicit access and compressed state. Full attention retains a separate key and value for every prior token. In contrast, linear attention, state-space models, and recurrent models summarize the prefix into a fixed-size or structured state. This shift is the main source of their efficiency, but it also creates the possibility of interference. If many similar facts are written into the same compressed memory, a later query may recover the wrong association or a value that was never present in the prompt.

The second theme is *forgetting*. Retention and recurrent models often use decay-like behavior to keep memory bounded and suppress stale information. Such decay is useful when most old tokens are irrelevant, but retrieval tasks expose the opposite case: an early token may remain essential even after many irrelevant updates. Long-context retrieval therefore tests whether a model can preserve a small piece of information without allowing it to fade or be overwritten.

The third theme is *selectivity*. Mamba’s central contribution is to make the compressed state’s update form depend on the input token, allowing the model to decide which information should be retained or suppressed [6]. Selectivity is important for retrieval because it gives the model a possible mechanism for filtering irrelevant context. However, selectivity does not remove the finite-state bottleneck: the model must still decide what to store and what to sacrifice.

The fourth theme is *overwrite* and *interference*. For example, Gated DeltaNet explicitly targets the problem that additive updates in its accumulated association memory can be too blunt when many similar associations are present [8]. Its gated delta-rule update is designed to edit memory more precisely along the current key direction. In the retrieval setting, this idea is directly relevant: distractor records create many nearly identical write operations, and the model must preserve the association requested by the final query without confusing it with the others.

Finally, hybrid architectures reflect a practical compromise. BASED, Mamba-2-Attn, Griffin, Jamba,

and Nemotron-H combine multiple sequence-mixing mechanisms in one architecture. Local or sparse attention can preserve some exact token access, while linear, recurrent, or state-space components provide cheaper global context processing. In retrieval terms, hybrid models ask whether a small number of explicit-access layers can protect key-value binding while most layers use cheaper compressed or local sequence mixing. This trend frames the long-context problem as a question of how much explicit access must be retained for a given task.

5 Empirical Study Overview

The survey above covers a wider landscape than can be reliably tested in a small reproduction study. A meaningful empirical comparison requires more than architectural variety: the tested checkpoints should be public, loadable under a common evaluation pipeline, and reasonably comparable in scale, tokenizer, training conditions, and implementation quality. Otherwise, differences in pretraining data, prompt format, numerical kernels, or checkpoint provenance can be mistaken for architectural effects.

The empirical portion of this report therefore uses two complementary public checkpoint families. The first is the HazyResearch family released with the BASED paper, which includes attention, Mamba, and BASED models at two smaller scales. It provides a broad matched-family contrast between full attention, state-space memory, and hybrid linear attention under a hard distractor stress test. The second is the official State Spaces Mamba family, comparing Mamba 2.8B, Mamba-2 2.7B, and Mamba-2-Attn 2.7B. It provides a narrower and more graded comparison within the Mamba family using intermediate distractor counts. Together, these experiments address three questions. First, how do explicit attention, state-space memory, and hybrid efficient architectures behave when synthetic retrieval requires exact key-value binding? Second, does Mamba-2 improve retrieval robustness relative to original Mamba under similar public checkpoint conditions? Third, how much do the conclusions depend on the interface used to elicit or score an answer?

Both experiments use synthetic passkey retrieval. A prompt contains one or more records of the form

`PASSKEY_RECORD[key] = value.`

At the end of the document, the model is asked to complete the matching record for a queried key. The primary metric is exact-match generation accuracy: the first number generated by the model is compared to the target value. We also record failure type. A distractor-value error means the model produced the value of another record in the document; a not-in-document error means it produced a number not present in the prompt; and a no-number error means it failed to emit a numeric answer.

For the Mamba-family experiment, we also use a likelihood diagnostic. Instead of allowing the model to generate freely, the diagnostic ranks candidate numeric continuations after the same passkey-completion prompt. The candidate set is the target value plus all distractor values, or synthetic numeric negatives when no distractors are present. This diagnostic helps distinguish between failure to identify the answer and failure to emit it cleanly. It is used as a companion to generation accuracy, not as a replacement.

The experiments should be interpreted as controlled probes of retrieval behavior in available public models. They are not universal architectural rankings, and they do not reproduce the throughput or memory-efficiency claims of the original papers.

6 Experiment 1: BASED-Family Retrieval

The first experiment evaluates the HazyResearch checkpoint family released with the BASED paper. This family is useful because it contains an attention baseline, a Mamba baseline, and a BASED model at two similar scales. Full attention preserves explicit pairwise access to earlier tokens. Mamba replaces that access with a selective state-space recurrence, compressing the past into a structured hidden state. BASED combines efficient sequence-mixing mechanisms, including linear attention, to target a recall-throughput tradeoff. The comparison therefore tests whether the expected retrieval advantage of explicit attention appears when the checkpoint source and tokenizer family are held relatively fixed.

Model label	Architecture	Checkpoint	Evaluation note
Attention 360M	Full attention	hazyresearch/attn-360m	bf16, cached decoding
Attention 1.4B	Full attention	hazyresearch/attn-1b	bf16, cached decoding
Mamba 360M	Selective SSM	hazyresearch/mamba-360m	bf16, cached decoding
Mamba 1.4B	Selective SSM	hazyresearch/mamba-1b	bf16, cached decoding
BASED 360M	Hybrid linear attention	hazyresearch/based-360m	fp32, recompute decoding
BASED 1.4B	Hybrid linear attention	hazyresearch/based-1b	fp32, recompute decoding

Table 2: Public HazyResearch checkpoints used in Experiment 1. The BASED models are run in fp32 with recompute decoding for accuracy reliability, as discussed in Section 6.4.

The experiment should be read as an accuracy-focused extension of the BASED model comparison, not as a reproduction of the paper’s throughput results. The optimized kernel path needed for faithful speed comparison was outside the scope of this implementation, and the BASED models use a conservative recompute decoding strategy after the reliability checks described below. Consequently, the reported outcomes are retrieval accuracy and failure type, not runtime or memory efficiency.

6.1 Tasks and Evaluation

Two synthetic retrieval tasks are used. The first is a single-needle task: one key-value record is placed in an otherwise irrelevant context, and the model is asked to return the value associated with the target key. The second is a multi-key task with distractors: the context contains the target record plus twenty same-format distractor records, and the model must return the value belonging to the queried key. The second task is harder because the model must not only notice the record format, but bind the correct key to its value among many similar alternatives.

The 20-distractor setting should be interpreted as a hard stress test rather than as a fine-grained degradation curve. It was chosen to make interference visible in a compact matrix across six models and eight lengths. The later State Spaces experiment uses 0, 5, and 10 distractors to resolve more gradual degradation after this harder setting showed that high distractor pressure can quickly saturate some efficient models.

Task	Distractors	Context lengths	Samples and scoring
Single needle	0	512, 768, 1024, 1280, 1536, 1792, 2048, 2560	30 examples per length; exact match on the first generated number
Multi-key retrieval	20	512, 768, 1024, 1280, 1536, 1792, 2048, 2560	30 examples per length; exact match on the first generated number

Table 3: Final evaluation matrix. With six models, two tasks, eight context lengths, and thirty examples per setting, the clean sweep contains 2880 model generations.

Datasets were generated deterministically and tokenized against the GPT-2 tokenizer used by the checkpoints. All models were evaluated on the same dataset files, prompts, target lengths, and scoring logic. Generation was greedy, with a maximum of eight new tokens. A prediction was counted as correct when the first number emitted by the model exactly matched the target value. Incorrect responses were further categorized as a distractor value, a number not present in the document, or a response with no generated number.

6.2 Single-Needle Results

Single-needle retrieval is comparatively forgiving. As shown in Figure 1 and Table 4, most models remain accurate across the tested context lengths. Attention 360M is perfect at all lengths except 768, where it scores 29/30. Attention 1.4B is less smooth at shorter lengths but reaches 30/30 at 2560. The 1.4B Mamba and both BASED models are also strong, while Mamba 360M shows the clearest late-context degradation, falling to 19/30 at 2560.

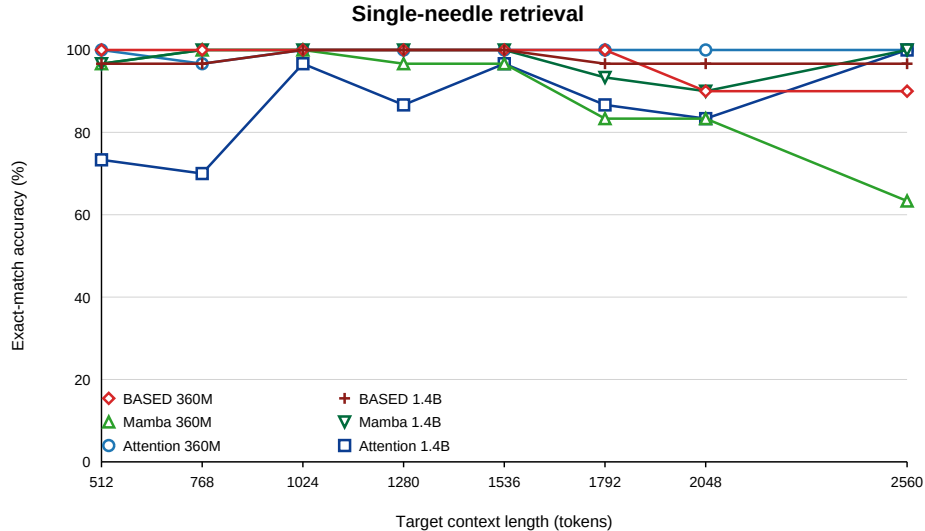


Figure 1: Exact-match accuracy on the single-needle task. Most models solve the task reliably, indicating that a single salient key-value record is not enough to sharply distinguish the architectures in this setting.

Model	512	768	1024	1280	1536	1792	2048	2560
Attention 360M	100	96.7	100	100	100	100	100	100
Attention 1.4B	73.3	70.0	96.7	86.7	96.7	86.7	83.3	100
Mamba 360M	96.7	100	100	96.7	96.7	83.3	83.3	63.3
Mamba 1.4B	96.7	100	100	100	100	93.3	90.0	100
BASED 360M	100	100	100	100	100	100	90.0	90.0
BASED 1.4B	96.7	96.7	100	100	100	96.7	96.7	96.7

Table 4: Single-needle exact-match accuracy by target context length (%). Each cell summarizes 30 examples.

The main implication is that simple passkey retrieval can overstate a model’s long-context retrieval ability. When the context contains only one relevant record, even compressed-memory architectures often have enough signal to preserve or reconstruct the target association.

6.3 Multi-Key Results

The multi-key task produces a much stronger separation. Figure 2 and Table 5 show that the attention baselines retain substantially higher accuracy than Mamba or BASED when twenty same-format distractors are present. Attention 1.4B is the strongest model through 2048 tokens, scoring between 70.0% and 96.7%, while Attention 360M remains moderate and declines to 23.3% at 2560.

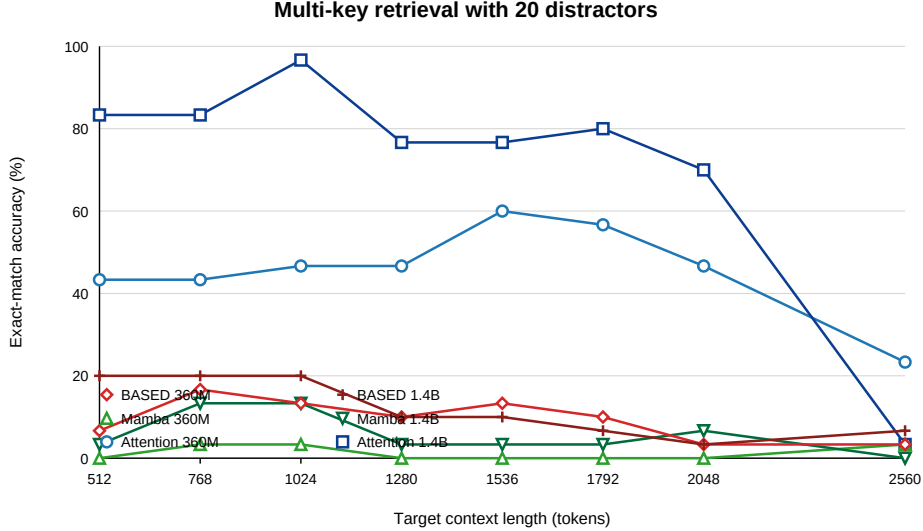


Figure 2: Exact-match accuracy on the multi-key task with twenty distractor records. The distractor setting separates the full-attention baselines from the compressed-memory models much more clearly than the single-needle task.

Model	512	768	1024	1280	1536	1792	2048	2560
Attention 360M	43.3	43.3	46.7	46.7	60.0	56.7	46.7	23.3
Attention 1.4B	83.3	83.3	96.7	76.7	76.7	80.0	70.0	3.3
Mamba 360M	0	3.3	3.3	0	0	0	0	3.3
Mamba 1.4B	3.3	13.3	13.3	3.3	3.3	3.3	6.7	0
BASED 360M	6.7	16.7	13.3	10.0	13.3	10.0	3.3	3.3
BASED 1.4B	20.0	20.0	20.0	10.0	10.0	6.7	3.3	6.7

Table 5: Multi-key exact-match accuracy by target context length (%). Each cell summarizes 30 examples.

Mamba and BASED perform poorly in this distractor-heavy setting. Their errors are not usually blank responses; the models often generate either a distractor value or a number not present in the document. This is consistent with the retrieval risk described earlier: when many similar records are written into compressed state, the model may preserve the general task format without retaining the exact queried binding.

Model	Correct	Distractor	Not in document	No number
Attention 360M	23.3	53.3	23.3	0.0
Attention 1.4B	3.3	3.3	0.0	93.3
Mamba 360M	3.3	20.0	76.7	0.0
Mamba 1.4B	0.0	50.0	50.0	0.0
BASED 360M	3.3	23.3	73.3	0.0
BASED 1.4B	6.7	33.3	60.0	0.0

Table 6: Failure categories on the 2560-token multi-key setting (%). The Attention 1.4B collapse is qualitatively different from the compressed-memory failures because it usually generates no numeric answer at all.

6.4 Diagnostic Findings

BASED decoding reliability. A reliability check on the BASED implementation found that cached generation could create an implausible nonmonotonic collapse in the single-needle task: accuracy dropped

sharply at an intermediate length and then recovered at longer lengths. This pattern did not resemble ordinary degradation with context length. A targeted diagnostic compared cached generation against recompute decoding, where the full forward pass is rerun for each new token. On the previously failing 1792-token single-needle setting, BASED 360M in fp32 improved from 2/20 under cached generation to 20/20 under recompute decoding. The final matrix therefore uses recompute decoding for BASED models.

This choice is important for interpretation. It improves the reliability of the accuracy results, but it also means that runtime measurements from these runs should not be compared against the attention or Mamba cached-generation paths. For this reason, timing and memory measurements are not analyzed as experimental outcomes.

Attention 1.4B multi-key collapse. The most surprising result in the final matrix is Attention 1.4B at the 2560-token multi-key setting. The model falls from 21/30 at 2048 tokens to 1/30 at 2560, while the same checkpoint scores 30/30 on the 2560-token single-needle task. This suggests that the failure is not a hard context-window limit. Additional probes reproduced the collapse on the same thirty examples: increasing the generation budget, using recompute decoding, making the instruction more explicit, and allowing an explicit unknown sentinel did not restore performance. Instead, the model usually resumed document-style continuation and generated no numeric answer. The result is therefore treated as a stable task-mode collapse in this prompt setting, not as clean evidence of an architectural limit of attention.

7 Experiment 2: State Spaces Mamba Family

The second experiment focuses on larger official State Spaces checkpoints: `state-spaces/mamba-2.8b`, `state-spaces/mamba2-2.7b`, and `state-spaces/mamba2attn-2.7b`. These models are similar in scale and come from the same public release family, documented as trained on 300B tokens on the Pile. This does not make the comparison a perfect architectural ablation, since exact low-level training details are not fully audited here. It provides a cleaner setting for asking whether Mamba-2 improves retrieval robustness relative to original Mamba.

Original Mamba uses selective state-space layers to update a compressed sequence state as the prompt is processed. Mamba-2 reformulates this family through structured state-space duality, preserving the efficient recurrent view while improving the architecture’s parallel and hardware-friendly properties. Mamba-2-Attn is a hybrid variant that adds attention blocks to the Mamba-2 family. It is therefore useful for asking whether adding some explicit token access improves retrieval over a purely state-space model at similar scale.

The corresponding State Spaces Transformer++ checkpoint was also considered, but not used as a primary baseline. Under the passkey-completion interface, it showed severe task-mode collapse: the model often resumed document-style continuation rather than emitting a numeric answer. Because that behavior did not provide a stable retrieval baseline, it is treated as a diagnostic observation rather than as an architectural comparison.

Model label	Checkpoint	Role in comparison
Mamba 2.8B	<code>state-spaces/mamba-2.8b</code>	Original selective state-space model at comparable scale.
Mamba-2 2.7B	<code>state-spaces/mamba2-2.7b</code>	Main successor architecture using the Mamba-2 formulation.
Mamba-2-Attn 2.7B	<code>state-spaces/mamba2attn-2.7b</code>	Hybrid variant that adds attention blocks to the Mamba-2 family.

Table 7: Official State Spaces checkpoints used in Experiment 2. The models are similar in scale and evaluated with the same dataset, prompt format, and scoring logic.

The task uses the same passkey-record format as Experiment 1, but with a different matrix. We evaluate context lengths of 2048, 4096, and 8192 tokens, with 0, 5, or 10 distractors and 30 examples

per setting. The full generation matrix contains 810 model generations. This design avoids the all-or-nothing effect of the earlier 20-distractor probe while still increasing binding pressure beyond single-needle retrieval.

7.1 Generation Results

Figure 3 and Table 8 show exact-match generation accuracy. Mamba-2 is the most robust model overall. It is perfect without distractors, strong at 2048 and 4096 with 5 or 10 distractors, and degrades mainly in the hardest 8192-token, 10-distractor setting. Original Mamba performs well at shorter no-distractor settings but collapses at 8192. Mamba-2-Attn is strong at 2048 but falls sharply once both context length and distractor pressure increase.

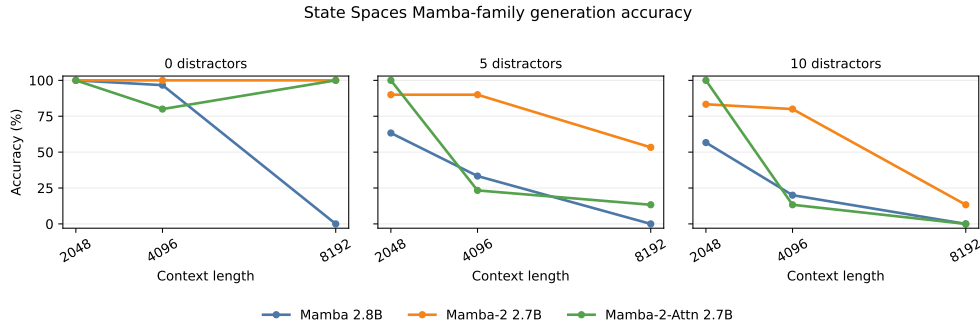


Figure 3: Generation accuracy for the State Spaces Mamba-family experiment. Mamba-2 remains strongest across the matrix, while Mamba and Mamba-2-Attn degrade more sharply under length and distractor pressure.

Distractors	Model	2048	4096	8192
0	Mamba 2.8B	100.0	96.7	0.0
0	Mamba-2 2.7B	100.0	100.0	100.0
0	Mamba-2-Attn 2.7B	100.0	80.0	100.0
5	Mamba 2.8B	63.3	33.3	0.0
5	Mamba-2 2.7B	90.0	90.0	53.3
5	Mamba-2-Attn 2.7B	100.0	23.3	13.3
10	Mamba 2.8B	56.7	20.0	0.0
10	Mamba-2 2.7B	83.3	80.0	13.3
10	Mamba-2-Attn 2.7B	100.0	13.3	0.0

Table 8: State Spaces generation accuracy by distractor count and target context length (%). Each cell summarizes 30 examples.

The failure types are also informative. Mamba’s 8192-token failures often produce no usable number or produce nonsensical numeric fragments, suggesting that the model is not merely selecting the wrong record. Mamba-2 failures at the hardest settings are usually distractor values, which points more directly to key-value binding confusion. Mamba-2-Attn also fails mostly through distractor selection once distractors and longer contexts are combined.

7.2 Completion-Likelihood Diagnostic

Because base models are sensitive to prompt interface, we add a numeric-likelihood diagnostic that preserves the generation prompt ending,

PASSKEY_RECORD[key] =,

but scores candidate numeric continuations rather than allowing free generation. Candidates are ranked by average token log-likelihood. This keeps the task shape fixed while asking whether the model assigns the highest probability to the correct value among plausible alternatives.

A less aligned natural-language **Question: ... Answer:** probe produced much lower distractor accuracy, especially for Mamba-2, and appeared to measure prompt-format mismatch as much as retrieval. This sensitivity is consistent with broader long-context evaluation practice: benchmarks such as RULER use fixed templates and answer prefixes because output format can contaminate retrieval scores [14], and prior work has shown that long-context performance can depend on prompt placement and structure [15].

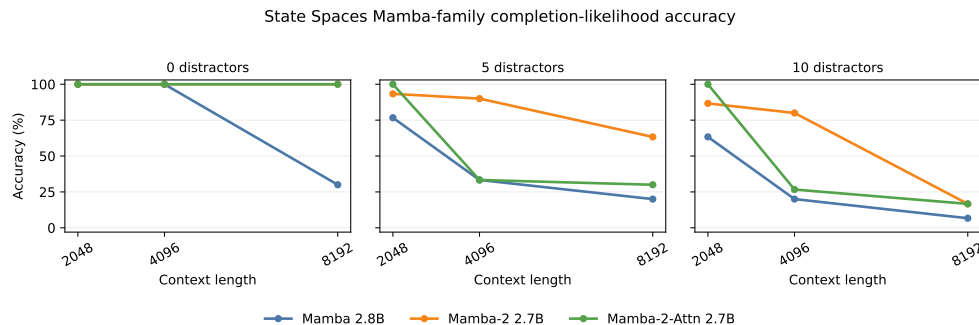


Figure 4: Completion-likelihood accuracy for the State Spaces Mamba-family experiment. The diagnostic uses the same passkey-completion prompt as generation, but ranks candidate numeric continuations by average log-likelihood.

Distractors	Model	2048	4096	8192
0	Mamba 2.8B	100.0	100.0	30.0
0	Mamba-2 2.7B	100.0	100.0	100.0
0	Mamba-2-Attn 2.7B	100.0	100.0	100.0
5	Mamba 2.8B	76.7	33.3	20.0
5	Mamba-2 2.7B	93.3	90.0	63.3
5	Mamba-2-Attn 2.7B	100.0	33.3	30.0
10	Mamba 2.8B	63.3	20.0	6.7
10	Mamba-2 2.7B	86.7	80.0	16.7
10	Mamba-2-Attn 2.7B	100.0	26.7	16.7

Table 9: Completion-likelihood accuracy by distractor count and target context length (%). Each cell summarizes 30 examples.

The completion-likelihood diagnostic supports the generation interpretation. Mamba-2 remains the strongest model, and its 2048- and 4096-token distractor results are close to its generation accuracy. Mamba-2-Attn’s 4096-token zero-distractor generation dip disappears under likelihood scoring, indicating that the correct value was still ranked highest and the generation errors were likely formatting or near-miss artifacts. Original Mamba’s 8192-token collapse, however, remains visible: it scores only 30.0% with no distractors and 6.7% with 10 distractors under completion likelihood.

8 Cross-Experiment Discussion

The two experiments support several bounded conclusions. First, distractors are a stronger stressor than length alone in these passkey tasks. In the BASED-family comparison, most models solve single-needle retrieval, but performance separates sharply once the context contains many same-format records. This supports the mechanistic concern that compressed or linear memory can blur similar key-value bindings.

Second, the results should not be collapsed into a single claim that efficient sequence models either beat or fail to beat attention. The HazyResearch attention baselines provide evidence that explicit attention remains powerful for exact retrieval when a stable attention checkpoint is available. The State Spaces Transformer++ checkpoint, by contrast, did not produce a reliable passkey-completion baseline in our diagnostics. That result is a methodological warning about base-model evaluation, not evidence that attention is inherently weak.

Third, the Mamba-2 result is the clearest positive finding. Across generation and completion likelihood, Mamba-2 substantially outperforms original Mamba under similar official checkpoint conditions. The improvement is especially visible at 4096 tokens with distractors, where Mamba-2 remains near 80–90% while Mamba falls to 20–33%. At 8192 tokens with 10 distractors, however, even Mamba-2 falls sharply. The hardest setting therefore still exposes a real binding challenge: the model often selects a distractor value rather than the queried value.

Fourth, the hybrid results are mixed. BASED and Mamba-2-Attn both combine multiple sequence-mixing mechanisms, but neither hybrid uniformly solves retrieval under distractor pressure. Mamba-2-Attn is excellent at 2048 tokens, yet much less stable than pure Mamba-2 at 4096 and 8192 with distractors. This does not show that attention is harmful, nor does it contradict attention’s general usefulness in other settings. It shows only that adding attention in this hybrid checkpoint does not automatically produce better synthetic key-value retrieval under the tested prompt and distractor conditions.

Finally, prompt interface matters. The same candidate-likelihood method gave very different results when applied after an **Answer:** prompt versus after the original `PASSKEY_RECORD[key]` = completion prompt. For base models, a likelihood diagnostic should preserve the task interface whenever possible. Otherwise, it may measure whether the model understands a question-answer format rather than whether it can retrieve the target record.

9 Limitations

Several limitations qualify these conclusions. First, the tasks are synthetic. They isolate exact key-value binding under context length and distractor pressure, but they do not cover the full range of long-context reasoning, summarization, code analysis, or retrieval-augmented generation. Second, all tested models are public base checkpoints. Instruction tuning, different pretraining mixtures, or larger scales could change performance substantially.

Third, the experiments do not provide faithful throughput comparisons. The BASED runs use recompute decoding for accuracy reliability, and the optimized kernel path needed for speed evaluation was outside the scope of this implementation. As a result, the experiments speak to retrieval accuracy, not to the full recall-throughput tradeoff emphasized by the BASED paper.

Fourth, architecture is not perfectly isolated from training. The State Spaces models come from the same public release family and similar scale, but the public artifacts do not allow every low-level training hyperparameter to be audited. The findings are therefore best read as checkpoint-family evidence, not as a definitive causal ablation. Fifth, the distractor grids are not identical across experiments: the BASED-family experiment uses a 20-distractor stress test, while the State Spaces experiment uses intermediate distractor counts to examine partial degradation. Because these grids differ, absolute accuracy values should not be compared directly across Experiment 1 and Experiment 2; the cross-experiment comparison is qualitative and methodological rather than a single shared leaderboard. Finally, prompt format affects base-model evaluation. We address this with completion-style likelihood diagnostics, but the broader point remains: long-context retrieval benchmarks measure an interaction between model, prompt, decoding, and scoring.

10 Conclusion

This study supports a nuanced view of synthetic key-value retrieval in efficient language models. In the BASED-family experiment, attention baselines remain strongest when many same-format distractors are present, while Mamba and BASED often confuse the queried value with another record or with an

unsupported number. This result is consistent with the idea that compressed memory can struggle with exact key-value binding.

The State Spaces experiment shows a different but compatible result: within the Mamba family, Mamba-2 substantially improves retrieval robustness over original Mamba. The improvement appears both in free generation and in a prompt-aligned likelihood diagnostic. Mamba-2-Attn performs very well in short contexts but does not match pure Mamba-2 under longer distractor-heavy conditions.

The overall conclusion is therefore not that efficient sequence models simply succeed or fail at long-context retrieval. Rather, exact synthetic retrieval remains a hard test for compressed-memory architectures, especially when distractors create competing bindings. At the same time, newer state-space designs such as Mamba-2 can meaningfully improve robustness within that efficient modeling regime. Future work should connect these accuracy probes to faithful throughput measurements and to more natural long-context tasks.

References

- [1] A. Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.
- [2] M. Zaheer et al. Big Bird: Transformers for longer sequences. In *NeurIPS*, 2020.
- [3] Y. Sun et al. Retentive network: A successor to Transformer for large language models. arXiv preprint, 2023.
- [4] B. Peng et al. RWKV: Reinventing RNNs for the Transformer era. arXiv preprint, 2023.
- [5] M. Poli et al. Hyena hierarchy: Towards larger convolutional language models. In *ICML*, 2023.
- [6] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint, 2023.
- [7] T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *ICML*, 2024.
- [8] S. Yang et al. Gated Delta Networks: Improving Mamba2 with delta rule. arXiv preprint, 2024.
- [9] I. Schlag, K. Irie, and J. Schmidhuber. Linear Transformers are secretly fast weight programmers. In *ICML*, 2021.
- [10] S. Arora et al. Simple linear attention language models balance the recall-throughput tradeoff. arXiv preprint, 2024.
- [11] S. De et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. arXiv preprint, 2024.
- [12] O. Lieber et al. Jamba: A hybrid Transformer-Mamba language model. arXiv preprint, 2024.
- [13] NVIDIA et al. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. arXiv preprint, 2025.
- [14] C.-P. Hsieh et al. RULER: What’s the real context size of your long-context language models? arXiv preprint arXiv:2404.06654, 2024.
- [15] N. F. Liu et al. Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 2024.